

UNIVERSIDADE FEDERAL DE GOIÁS

INSTITUTO DE INFORMÁTICA

BACHARELADO EM SISTEMAS DE INFORMAÇÃO

Residência Técnica em Sistemas de Informação

Mapeamento do Ciclo DevOps e Esteira CI/CD

SQUAD 04

Versão	Data	Autor
1.0	27/06/2026	MATHEUS PAMPLONA OLIVEIRA

SUMÁRIO

1 - Objetivo.....	3
2 - Visão Geral do Ciclo DevOps e Modelo de Ramificação (GitFlow).....	3
3 - Arquitetura da Esteira de Integração e Entrega Contínua (CI/CD).....	3
3.1 - Pipeline de Integração Contínua (CI).....	4
3.2 - Pipeline de Entrega Contínua (CD).....	4
4 - Estratégia de Implantação de Alta Disponibilidade (Zero Downtime).....	4
5 - Infraestrutura como Código e Isolamento de Ambientes.....	5
5.1 - Dockerfile (Multi-Stage Build - Front-end).....	5
5.2 - Orquestração Local (Docker Compose).....	5
6 - Gerenciamento Avançado de Segredos e Configurações (Secret Management).....	6
7 - Monitoramento, Logs e Observabilidade Operacional.....	6
8 - Protocolo de Onboarding e Passagem de Bastão.....	6

1 - Objetivo

Este documento estabelece as diretrizes arquiteturais, operacionais e práticas de engenharia de software voltadas ao Mapeamento do Ciclo DevOps e à especificação da Esteira CI/CD (Continuous Integration / Continuous Deployment) para o ecossistema eFCaaS. Derivado da análise estrutural do repositório Squad-4-Main, este artefato funciona como um guia de infraestrutura indispensável para padronizar o ciclo de vida do desenvolvimento, mitigar falhas de ambiente e viabilizar uma passagem de bastão fluida e segura para as próximas equipes encarregadas da evolução do software.

2 - Visão Geral do Ciclo DevOps e Modelo de Ramificação (GitFlow)

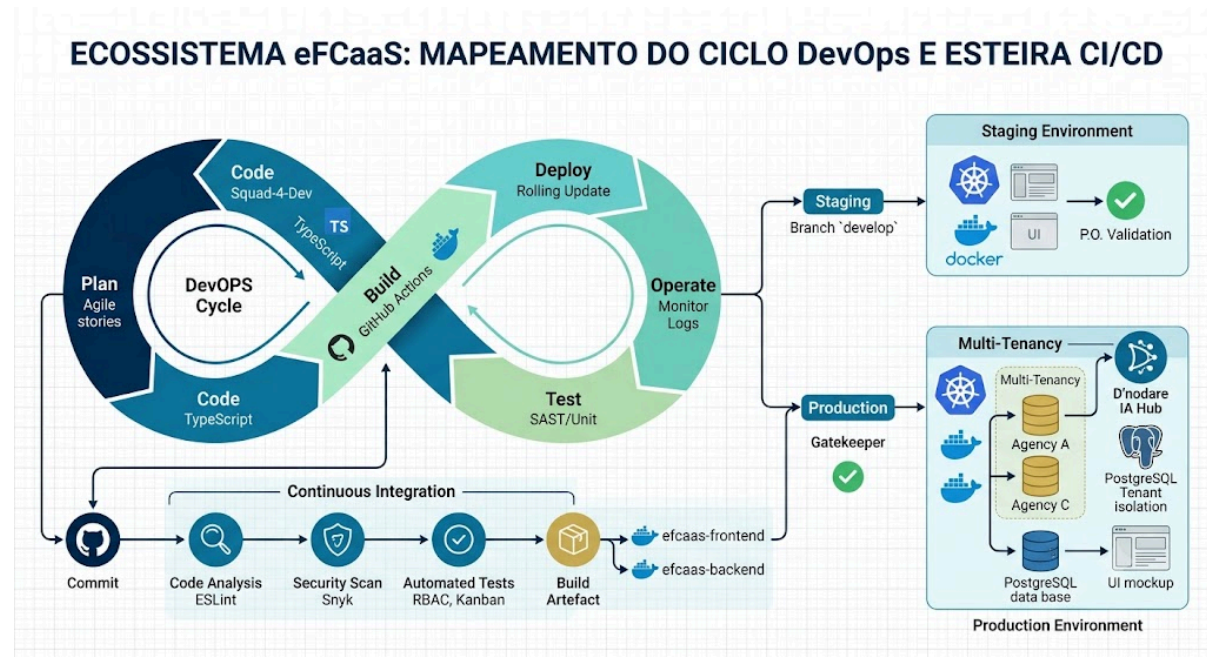
O ciclo de vida do eFCaaS foi projetado para sustentar uma arquitetura distribuída de microsserviços e garantir o isolamento essencial do modelo multi-tenant. Visando organizar o desenvolvimento paralelo conduzido pela equipe sem comprometer a estabilidade do código em produção, estabelece-se o uso rigoroso do fluxo de trabalho GitFlow baseado nas seguintes ramificações:

- **Branch main:** Centraliza o código em estado estritamente estável e homologado para produção. Cada incremento bem-sucedido nesta branch resulta no disparo de tags de versão formais (ex: v1.0.0).
- **Branch dev:** Linha de base contínua que reflete o estado consolidado da sprint atual. Funciona como o espelho direto do ambiente de Homologação, agregando as implementações prontas antes do deploy final.
- **Branches feature/*:** Ramificações temporárias abertas por desenvolvedores a partir da dev para o cumprimento de tarefas específicas e requisitos do roadmap (ex: feature/docs). Ao término, seu código passa por escrutínio via Pull Request (PR).

3 - Arquitetura da Esteira de Integração e Entrega Contínua (CI/CD)

A automação das etapas de validação e implantação baseia-se em gatilhos automáticos vinculados às ações de push e merge nas branches controladas.

Figura 1 : Ecosistema eFCaaS



Fonte: Elaborado pelo Squad 04 da Residência Técnica em Sistemas de Informação (2026/1).

Disponível em: < [Entregável 4 : Roadmap e DevOps](#) >. Acesso em: 27 jun. 2026.

3.1 - Pipeline de Integração Contínua (CI)

- **Análise Estática de Código (Linting):** Execução programada do ESLint e do Prettier no ecossistema efcas-frontend para forçar a aderência aos padrões de formatação TypeScript, eliminando bugs de sintaxe previamente.
- **Varredura Estática de Segurança (SAST):** Auditoria preventiva sobre a árvore de dependências (package-lock.json) por meio de ferramentas como npm audit ou Snyk, prevenindo a introdução de pacotes maliciosos ou desatualizados no ecossistema.
- **Execução de Testes Automatizados:** Validação compulsória de testes unitários e de integração (com foco na segurança do RBAC e transições de cards do Kanban). Falhas nestas etapas interrompem imediatamente o pipeline.
- **Compilação de Artefatos (Build):** Processamento e minificação do código-fonte através do script npm run build, gerando os entregáveis estáticos de alta performance da aplicação.

3.2 - Pipeline de Entrega Contínua (CD)

- **Empacotamento e Tagueamento Docker:** Compilação automatizada da imagem com base no Dockerfile e publicação subsequente em um repositório seguro de imagens (ex: GitHub Container Registry), identificada pelo hash do commit correspondente.

- **Deploy Automatizado em Homologação:** Gatilho disparado por atualizações na branch develop, publicando as modificações em ambiente de testes para homologação e validação contínua de interface e regras de negócio junto à Product Owner (P.O.).
- **Deploy Controlado em Produção:** Atualização automática do ambiente oficial das agências parceiras disparada a partir da main, condicionada à aprovação e assinatura manual (Gatekeeper) do mantenedor do repositório.

4 - Estratégia de Implantação de Alta Disponibilidade (Zero Downtime)

Dada a natureza crítica do eFCaaS no combate contínuo à desordem informacional, a esteira CD adota mecanismos de implantação progressiva baseados em estratégias de Rolling Update (ou infraestrutura Blue-Green via orquestrador). Isso permite que as novas instâncias de contêineres do front-end e back-end sejam provisionadas e testadas em segundo plano, ocorrendo a virada definitiva de tráfego de rede somente após a validação completa de saúde (Healthchecks) da nova versão. A abordagem zera o tempo de indisponibilidade da plataforma, resguardando as atividades operacionais dos jornalistas e checadores.

5 - Infraestrutura como Código e Isolamento de Ambientes

Com o intuito de neutralizar inconsistências ligadas à variação de sistemas operacionais locais, as definições de infraestrutura do eFCaaS foram abstraídas em arquivos declarativos de configuração.

5.1 - Dockerfile (Multi-Stage Build - Front-end)

```
# Estágio 1: Compilação da aplicação React
FROM node:18-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Estágio 2: Servidor de Produção de alta performance
FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

5.2 - Orquestração Local (Docker Compose)

```
version: '3.8'

services:
  efcaas-frontend:
    build: ./efcaas-frontend
    ports:
      - "3000:80"
    environment:
      - VITE_API_URL=http://localhost:8000
```

```

depends_on:
  - efcaas-backend

efcaas-backend:
  build: ./efcaas-backend
  ports:
    - "8000:8000"
  env_file:
    - .env
  depends_on:
    - efcaas-db

efcaas-db:
  image: postgres:15-alpine
  environment:
    POSTGRES_DB: efcaas_db
    POSTGRES_USER: admin
    POSTGRES_PASSWORD: securepassword_efcaas
  ports:
    - "5432:5432"
  volumes:
    - postgres_data:/var/lib/postgresql/data

volumes:
  postgres_data:

```

6 - Gerenciamento Avançado de Segredos e Configurações (Secret Management)

- A proteção dos dados e a integridade do modelo multi-tenant dependem da segregação rígida de credenciais sensíveis (como chaves privadas de integração com o D'nodare IA Hub, senhas de instâncias PostgreSQL e secrets de geração de tokens JWT).
- **Isolamento no Repositório:** O arquivo `.env` local contendo as chaves privadas encontra-se blindado no arquivo `.gitignore`, impedindo seu vazamento histórico em servidores públicos de código.
- **Injeção em Memória na Esteira:** Em ambientes oficiais (Homologação e Produção), os segredos são mantidos criptografados nos cofres de senhas da própria plataforma de CI/CD (ex: GitHub Secrets) e mapeados em variáveis de ambiente voláteis em memória apenas durante a inicialização dos contêineres, mitigando riscos de exposição.

7 - Monitoramento, Logs e Observabilidade Operacional

A operação pós-deploy é assistida por uma camada ativa de observabilidade, responsável por aferir a integridade da aplicação e a estabilidade das conexões remotas de IA:

- **Rastreabilidade de Falhas (Logs):** Consolidação centralizada de fluxos de erro gerados pelas camadas do back-end para detecção preditiva de anomalias em endpoints críticos.

- **Telemetria do Hub de IA:** Monitoria focada na latência média das respostas assíncronas e no volume de timeouts oriundos do processamento executado pelo D'nodare IA Hub, munindo os administradores com alertas acionáveis em cenários de degradação do serviço.

8 - Protocolo de Onboarding e Passagem de Bastão

Apoiada no arcabouço documental e nas configurações declarativas consolidadas pelo Squad 04, a próxima equipe técnica responsável pelo desenvolvimento pleno da solução deverá seguir o protocolo de inicialização abaixo:

1. **Clonagem de Repositório:** Realizar o download seguro da árvore de arquivos mapeada a partir do pacote Squad-4-Main.
2. **Configuração de Variáveis:** Replicar o arquivo descritivo `.env.example` localmente com o nome `.env`, preenchendo as chaves do banco de dados e os tokens de barramento da IA.
3. **Inicialização Local Unificada:** Chamar o terminal na raiz do projeto e rodar o comando `docker-compose up --build`. O Docker irá baixar as imagens do PostgreSQL, criar as tabelas locais isoladas, expor o back-end na porta 8000 e inicializar a interface React na porta 3000.
4. **Acoplamento de Esteira:** Registrar o fluxo de automação contido na documentação nas configurações de Actions do repositório oficial em nuvem cedido pela UFG, ativando as rotinas automáticas de deploy estruturadas com base no GitFlow.